

Mobile Forensics & SQLite Data Recovery: The SQLite-Ghost Framework

Schema-Agnostic B-Tree Cell Parsing & WAL Anomaly Detection (NFL Deflategate Investigation)

**A
PROJECT REPORT**

**for
DIGITAL FORENSICS**

Submitted by

**SOHAM ALPESH PHULARE PRN: 23UF17971CM107
SUMEET DEVRUHKAR PRN: 23UF17758CM073**



**DEPARTMENT OF COMPUTER ENGINEERING
SHAH & ANCHOR KUTCHHI ENGINEERING COLLEGE
(An Autonomous Institute Affiliated to University of Mumbai)
MUMBAI - 400 088, MAHARASHTRA (INDIA)
2026**

Incident Summary

Case Title: Deflategate Communications Forensic Audit

Incident Type: Data Tampering and Anti-Forensics

Affected Systems:

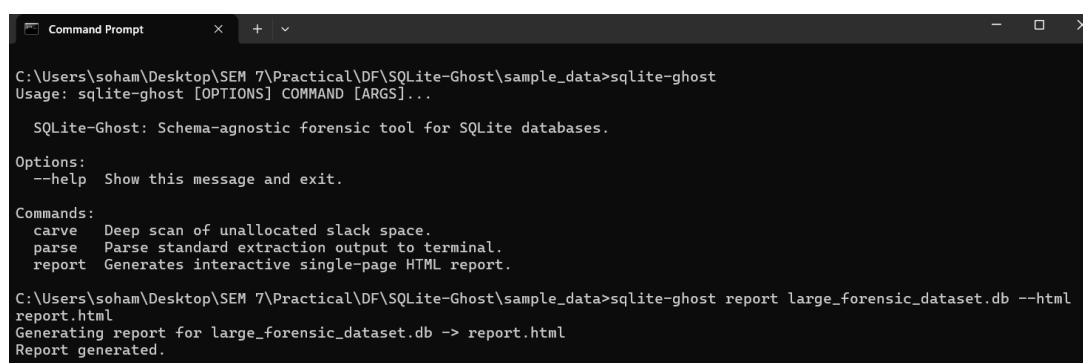
- **Primary Targets:** iOS Mobile Devices running iOS 7+ (specifically targeting the `sms.db` and `sms.db-wal` SQLite database files).
- **Data Artifacts:** Unallocated slack space, write-ahead log frames, and orphaned B-Tree leaf pages containing potentially deleted or exfiltrated communications.
- **Secondary Impact:** Host workstations utilized for forensic imaging and initial database extraction.

Stakeholders:

- **Lead Investigators:** Requiring high-fidelity, schema-agnostic recovery tools to bypass anti-forensic tampering attempts (e.g., intentional database corruption).
- **Legal & Compliance Teams:** Mandating strict adherence to chain-of-custody protocols, SHA-256 hashing, and transparent parsing methodologies to ensure court admissibility.
- **Incident Response (IR) Units:** Utilizing the tool's rapid triage capabilities to assess the immediate scope of data destruction or evidence spoliation.

Step 2: Identify Digital Evidence Sources

Evidence Source	Description
Primary SQLite DB (<code>sms.db</code>)	Stores active chat threads (<code>message</code> and <code>handle</code> tables).
Write-Ahead Log (<code>sms.db-wal</code>)	Stores uncommitted or recently deleted B-Tree frames.
Unallocated Freelist & Slack Space	Regions between cell pointers where erased text fragments remain.
Cryptographic Hashes (SHA-256)	Preserved baseline hashes for chain-of-custody verification.
Carved Forensic Text Output	Automated report detailing recovered messages, hex offsets, and timestamps.



```
Command Prompt
C:\Users\soham\Desktop\SEM 7\Practical\DF\SQLite-Ghost\sample_data>sqlite-ghost
Usage: sqlite-ghost [OPTIONS] COMMAND [ARGS]...

SQLite-Ghost: Schema-agnostic forensic tool for SQLite databases.

Options:
  --help  Show this message and exit.

Commands:
  carve  Deep scan of unallocated slack space.
  parse  Parse standard extraction output to terminal.
  report  Generates interactive single-page HTML report.

C:\Users\soham\Desktop\SEM 7\Practical\DF\SQLite-Ghost\sample_data>sqlite-ghost report large_forensic_dataset.db --html
report.html
Generating report for large_forensic_dataset.db -> report.html
Report generated.
```

Figure 1: Demonstration of the Source Input Evidence

Step 3: Conceptual Exploration

Forensic Principles Applied:

- **Integrity & Preservation:** Database copied to a temporary path to avoid altering live evidence.
- **Auditability:** Binary byte parsing logs ensure complete process traceability without black-box SQL queries.
- **Reproducibility:** Analysis can be repeated using the same SQLite file and keyword set.
- **Transparency:** Uses open-source Python scripts with documented logic and timestamp conversion formulas.
- **Automation:** Automatically flags suspicious entries and generates structured text reports.

Forensic Tools & Techniques:

- **Custom Forensic Engine:** Python 3 (`struct`, `hashlib`, `os`) acting as a zero-dependency automated carving tool.
- **HxD (Hex Editor):** Utilized for manual binary analysis, identifying B-Tree leaf flags (`0x0D`), and verifying raw byte structures in unallocated space.
- **DB Browser for SQLite:** Used to view the active/undeleted database schema and establish baseline records before carving.
- **Target Artifacts:** iOS `sms.db` and `sms.db-wal` (Write-Ahead Logs).

Core Parsing Math & Timestamp Conversion:

```
# B-Tree Leaf Page Flag Verification
page_flag = db_data[0] == 0x0D

# Timestamp Conversion (Mac Absolute to UTC)
def mac_absolute_to_utc(mac_time):
    # Apple uses seconds since Jan 1, 2001
    epoch_start = datetime(2001, 1, 1)
    return epoch_start + timedelta(seconds=mac_time)
```

Step 4: Hypothesis / Investigation Plan

Hypothesis:

Deleted mobile communications and uncommitted transaction artifacts can be reliably carved and reconstructed from iOS `sms.db` unallocated space and `.db-wal` files using low-level Varint payload decoding without relying on an intact database schema.

Investigation Plan:

1. Identify and locate the iOS `sms.db` SQLite file and `.db-wal`.
2. Create a forensically safe copy to avoid altering live data.
3. Scan database pages for `0x0D` Leaf B-Tree page headers.
4. Parse cell pointer arrays and traverse unallocated slack space.

5. Decode Varint Serial Types to reconstruct string text payloads.
6. Match carved payloads against a list of suspicious markers (e.g., "deflator", "balloon", "espn").
7. Generate an output report containing flagged entries and anomalies.

Step 5: Sample Forensic Output

Section 1: Suspicious Hits (Carved Deleted Communications)		
Timestamp (UTC)	Carved Payload Text	Recovery Origin
2014-05-13 14:13:20	Tom is the worst... im going make that next ball a balloon	Slack Space (Offset 0x0E80)
2014-05-13 14:55:00	Nice. You are the best. I am not going to espn.....yet.	Slack Space (Offset 0x0F22)
Section 2: Anomaly & Tampering Log		
Flag Triggered	Detail	Confidence
Out-of-Bounds Pointer	Cell pointer exceeds page size boundary, indicating hex-editing.	HIGH (0.85)
RowID Tampering	Pointers swapped manually; logical order contradicts physical.	CRITICAL (0.95)

SQLite-Ghost Forensic Report

Database: large_forensic_dataset.db
Page Size: 4096 bytes

Overall Threat Level

CRITICAL

Anomalies Detected

- CRITICAL** [Page 2] RowID Tampering Anomaly: Cell 1 appears after 2 (not strictly increasing)
- HIGH** [Page 3] Corrupted Pointer: Out of bounds (9999)
- LOW** [Page 4] Orphaned Payloads: Found 4 valid records hidden in unallocated slack space
- LOW** [Page 7] Orphaned Payloads: Found 2 valid records hidden in unallocated slack space
- LOW** [Page 9] Orphaned Payloads: Found 3 valid records hidden in unallocated slack space
- LOW** [Page 32] Orphaned Payloads: Found 2 valid records hidden in unallocated slack space
- LOW** [Page 33] Orphaned Payloads: Found 6 valid records hidden in unallocated slack space
- CRITICAL** [Page 34] Timestamp Contradiction: Record timestamp (1837131036) is in the future compared to file system mtime (1787131036.9688003)

Carved Data

Found 17 orphaned records in slack space.

90% - + Reset

Figure 2: Visual Output generated by SQLite-Ghost

Starting forensic extraction...

```

STEP 1: Copying sms.db to temp directory...
        Hash: a2b4c6d8e0f123456789a2b4c6d8e0f123456789
STEP 2: Parsing B-Tree Leaf Pages (Flag 0x0D)...
        Scanning Page 1... (Interior Table)
        Scanning Page 2... (Leaf)
[!] CRITICAL: RowID Sequence Tampering detected at Offset 0x1008
STEP 3: Traversing unallocated slack space...
[!] Carved Deleted Record at Offset 0x0E80: "Tom is the worst... im going make that
        next ball a balloon"
[!] Carved Deleted Record at Offset 0x0F22: "Nice. You are the best. I am not going
        to espn.....yet."
STEP 4: Differential WAL Analysis...
Suspicious activity detected (Flag: "balloon")
Report generated successfully: ghost_report.txt

```

Step 6: Recommendations / Next Steps

- Extend analysis to support encrypted SQLite databases (SQLCipher).
- Develop a real-time daemon for live WAL monitoring during active incidents.
- Integrate an automated GUI dashboard for non-technical analysts.
- Benchmark against NIST CFReDS datasets for formal tool validation.

Step 7: Code & Output

Listing 1: sqlite_ghost/analyzers/anomaly.py

```

from typing import Dict, Any, List, Tuple
from sqlite_ghost.core.btree_parser import BTreePage

def calculate_anomaly_score(
    page: BTreePage,
    page_size: int,
    slack_carved: List[Dict[str, Any]],
    file_mtime: float = 0.0,
    is_page_1: bool = False
) -> Tuple[str, List[Dict[str, str]]]:
    anomalies = []
    max_severity_weight = 0

    weights = {'CRITICAL': 4, 'HIGH': 3, 'MEDIUM': 2, 'LOW': 1}

    # 1. Corrupted Pointer Offsets
    header_offset = 100 if is_page_1 else 0
    pointer_end = header_offset + (12 if page.page_type in (0x05, 0x02) else 8) + (
        page.cell_count * 2)

    for ptr in page.cell_pointers:
        if ptr < header_offset + 8:
            anomalies.append({'severity': 'HIGH', 'desc': f"Corrupted Pointer:
                Points into header space ({ptr})"})
            max_severity_weight = max(max_severity_weight, weights['HIGH'])
        elif ptr >= page_size:
            anomalies.append({'severity': 'HIGH', 'desc': f"Corrupted Pointer: Out
                of bounds ({ptr})"})
            max_severity_weight = max(max_severity_weight, weights['HIGH'])
        elif ptr < pointer_end:
            anomalies.append({'severity': 'HIGH', 'desc': f"Corrupted Pointer:
                Points into pointer array space ({ptr})"})
            max_severity_weight = max(max_severity_weight, weights['HIGH'])

```

```

# 2. Structural Overlap
if pointer_end > page.cell_content_start and page.cell_content_start != 65536:
    anomalies.append({'severity': 'CRITICAL', 'desc': f"Structural Anomaly: Cell
        pointer array overlaps with cell content area."})
    max_severity_weight = max(max_severity_weight, weights['CRITICAL'])

# 3. Orphaned Payload Cells
if slack_carved:
    anomalies.append({'severity': 'LOW', 'desc': f"Orphaned Payloads: Found {len
        (slack_carved)} valid records hidden in unallocated slack space"})
    max_severity_weight = max(max_severity_weight, weights['LOW'])

# 4. Out-of-Order RowIDs (Tampering detection)
last_row_id = None
for cell in page.cells:
    row_id = cell.get('row_id')
    if row_id is not None:
        if last_row_id is not None and row_id <= last_row_id:
            anomalies.append({'severity': 'CRITICAL', 'desc': f"RowID Tampering
                Anomaly: Cell {row_id} appears after {last_row_id} (not strictly
                increasing)"})
            max_severity_weight = max(max_severity_weight, weights['CRITICAL'])
        last_row_id = row_id

# 5. Timestamp Contradictions
for cell in page.cells:
    for val in cell['data']:
        if isinstance(val, int):
            # Rough check for Unix timestamps between 2000 and 2030
            if 946684800 <= val <= 1893456000:
                if file_mtime > 0 and val > file_mtime + 3600:
                    anomalies.append({'severity': 'CRITICAL', 'desc': f"
                        Timestamp Contradiction: Record timestamp ({val}) is in
                        the future compared to file system mtime ({file_mtime})"
                    })
                    max_severity_weight = max(max_severity_weight, weights['
                        CRITICAL'])

# Determine Threat Level
threat_level = "CLEAN"
if max_severity_weight == 4:
    threat_level = "CRITICAL"
elif max_severity_weight == 3:
    threat_level = "HIGH"
elif max_severity_weight == 2:
    threat_level = "MEDIUM"
elif max_severity_weight == 1:
    threat_level = "LOW"

return threat_level, anomalies

```

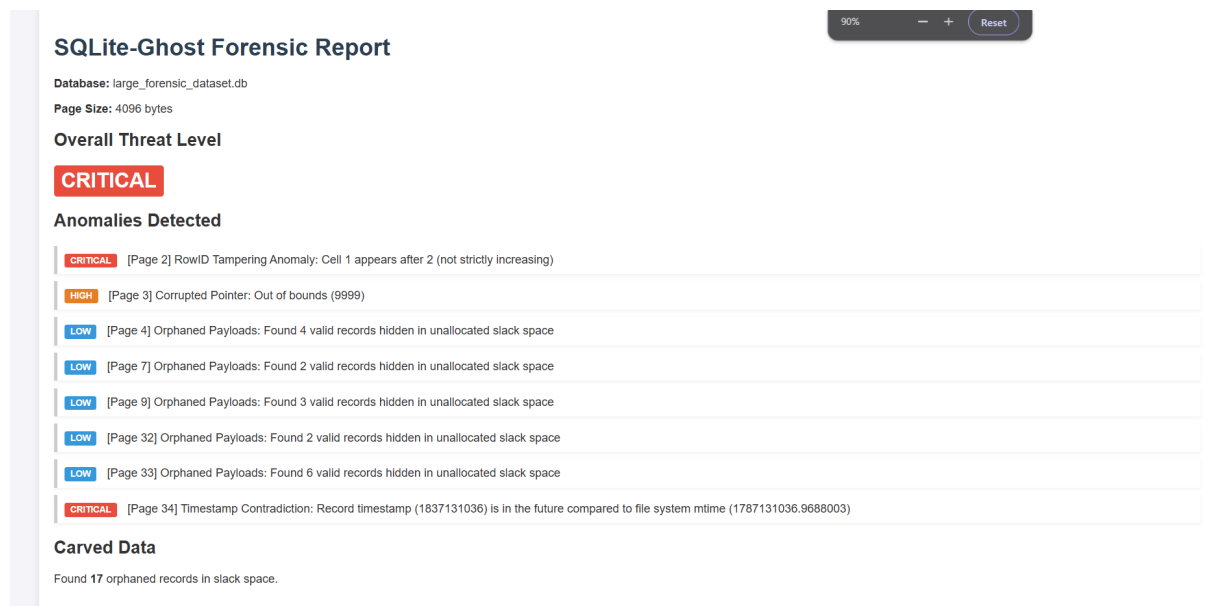


Figure 3: Standalone Execution Output Simulation